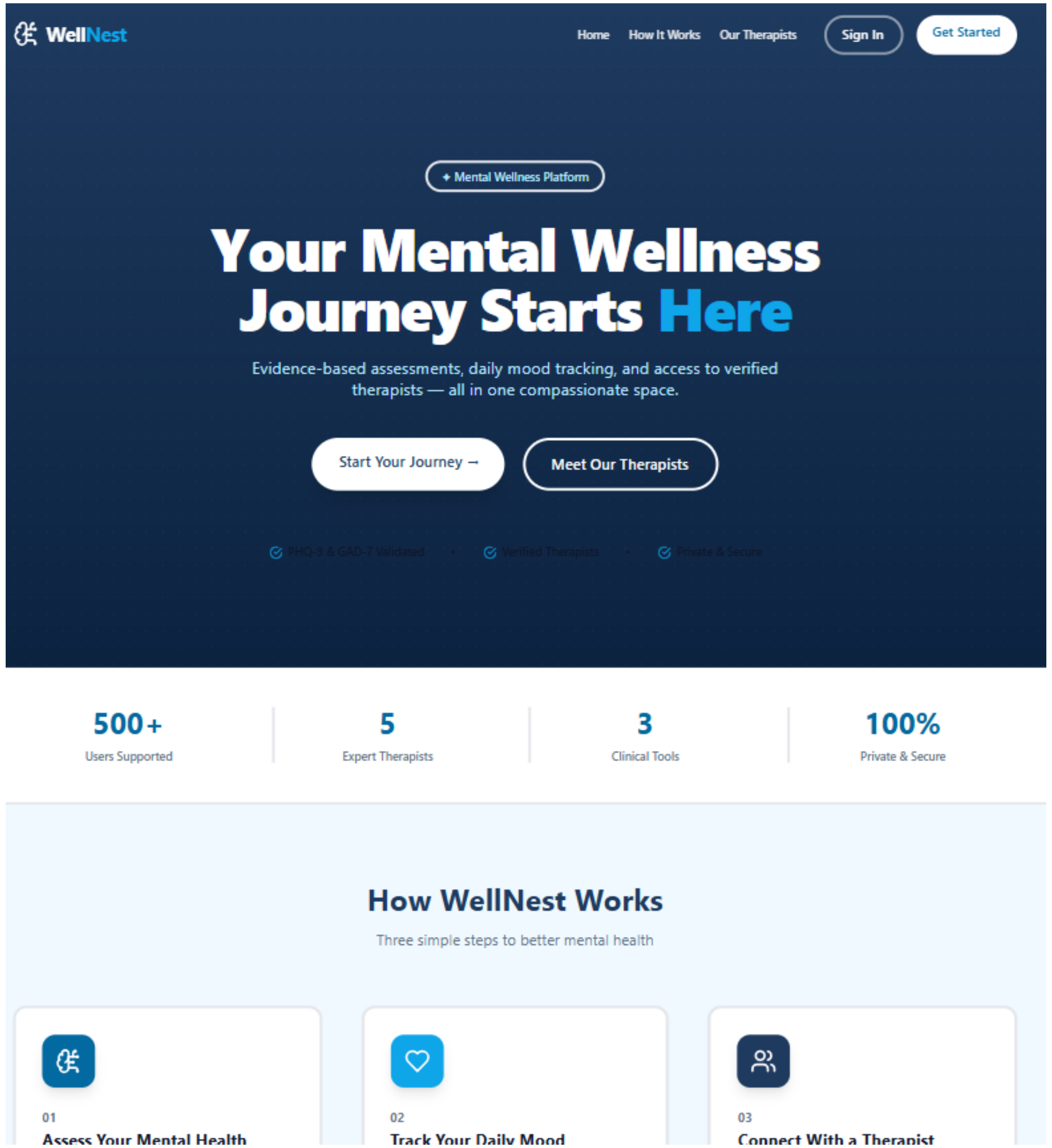
DEVOPS PROJECT

WellNest

A Cloud-Native Mental Health Platform — built on microservices, containerized with Docker, orchestrated on Kubernetes, and secured end-to-end with DevSecOps practices.

DEVOPS | DEVSECOPS | KUBERNETES | CI/CD | GITOPS



What is WellNest?

WellNest is a mental health platform built as **3 backend microservices + 1 frontend**, containerized with Docker, and orchestrated on Kubernetes across isolated dev and prod environments.

Auth Service

- User registration & login
- JWT tokens
- bcrypt password hashing
- MongoDB for user data

Assessment Service

- Mental health assessments
- PHQ-9 & GAD-7 questionnaires
- Score calculation & history
- MongoDB storage

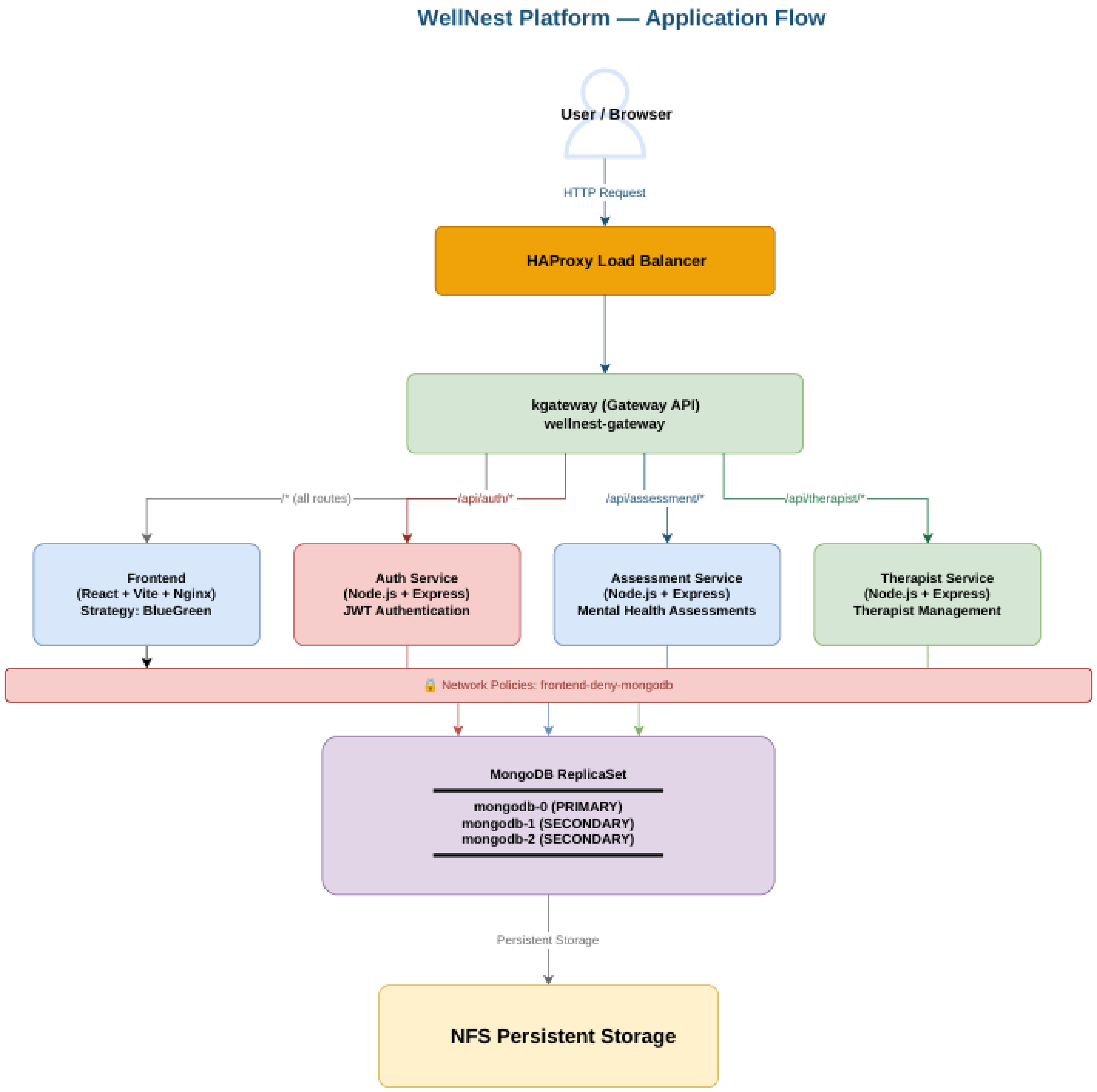
Therapist Service

- Therapist profiles & search
- Appointment booking
- Availability management
- MongoDB storage

Frontend

- React 18 + TypeScript + Vite
- Tailwind CSS + shadcn/ui
- JWT in localStorage, protected routes
- Served via Nginx

Application-Flow Diagram:



Containerization with Docker

Backend Services — Node.js Alpine

01

Base Image

`node:20-alpine` — minimal footprint via Alpine Linux

02

Install & Copy

`npm install --omit=dev` · `COPY src/`

03

Non-Root Security

`adduser -S wellnest` · `chown -R` · `USER wellnest`

04

Serve

`EXPOSE 3001/3002/3003` · `CMD: node src/index.js`

Frontend — Multi-Stage Build

1

Stage 1 — Builder

`node:20-alpine` · `npm install` · `npm run build` (Vite) → produces `/app/dist`

2

Stage 2 — Runtime

`nginx:alpine` · copy `dist/` · copy `nginx.conf`

3

Runtime Env Injection

Nginx serves static build. API URLs configured via env variables at build time.

Image Tagging Strategy: Dev: Short SHA (`3a4b5c6`) · Prod: Semantic Version (`v1.0.1`) · All images pushed to Docker Hub

Kubernetes — Resources & Architecture

Deployments

auth · assessment · therapist · frontend(Rollouts)

StatefulSets

MongoDB 7 — 3 replicas with persistent storage

PVCs

NFS StorageClass · 2Gi (dev) / 5Gi (prod) per MongoDB instance

NetworkPolicy

Deny Frontend to the MongoDB

Gateway (kgateway)

`/api/auth` → 3001 · `/api/assessment` → 3002 · `/api/therapist` → 3003 · `/` → frontend:80

Namespaces

wellnest-dev · wellnest-prod

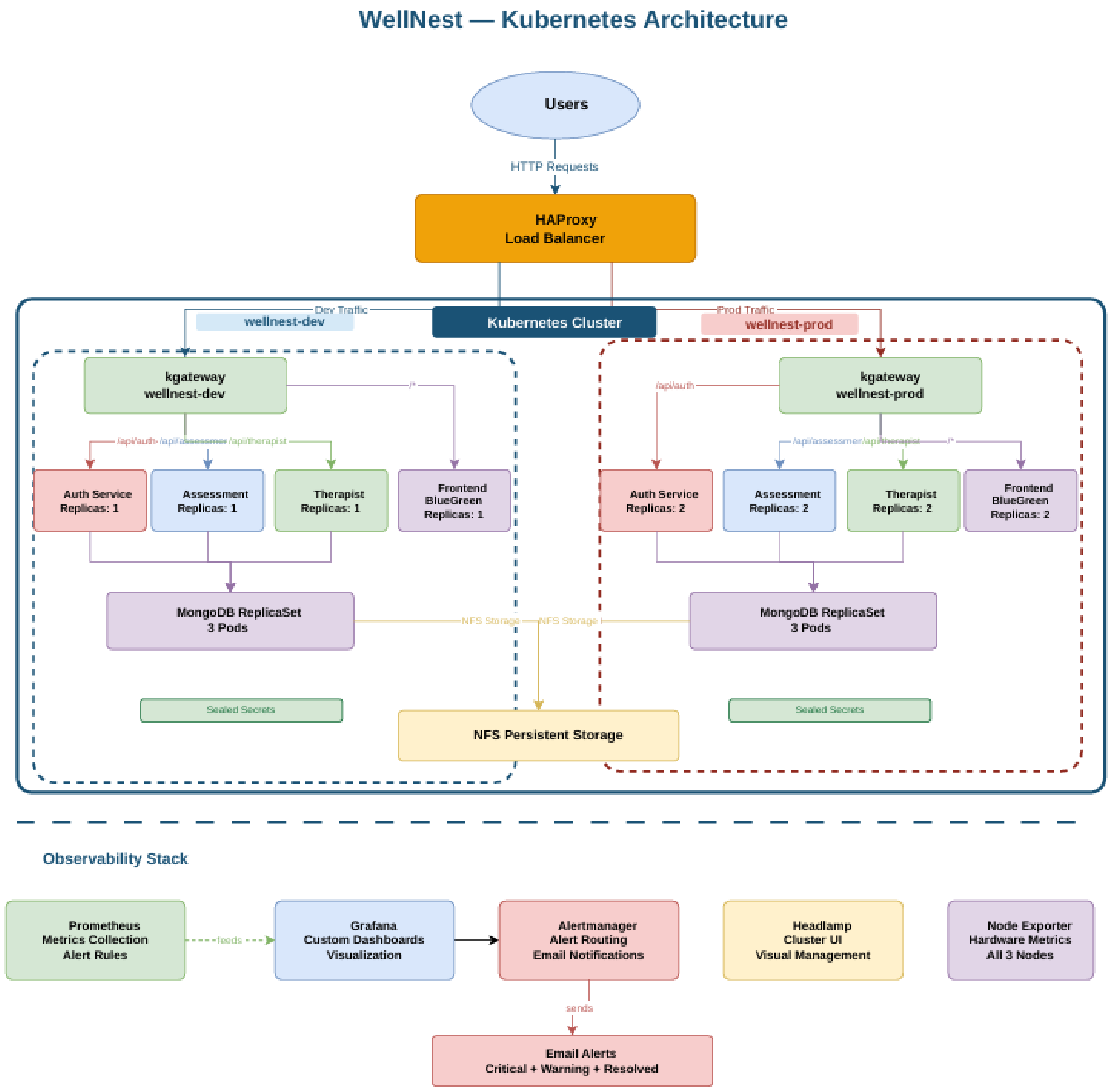
Dev Environment

`wellnest-dev` namespace · `develop` branch · `values-dev.yaml` · HPA min:1 max:3

Prod Environment

`wellnest-prod` namespace · `main` branch · `values-prod.yaml` · HPA min:2 max:5

Kubernetes Architecture — Traffic Flow



Traffic enters via HAProxy, is routed through kgateway using path-based rules, distributed across namespace-isolated microservices, and persisted to replicated MongoDB instances backed by NFS storage.

CI/CD — GitHub Actions, GitOps & DevSecOps

Dev Branch — Continuous Deployment

SAST

SonarQube static analysis with quality gate enforcement

SCA

Snyk dependency scan — blocks known vulnerable packages

Container Scan

Trivy image scan — CRITICAL + HIGH severity findings block the pipeline

GitOps Update

Bot updates values-dev.yaml and ArgoCD autosyncs.

Merge → build `{SHA}` → push Docker Hub → ArgoCD auto-syncs ✅

Main / Prod Branch — Continuous Delivery

SAST

SonarQube static analysis with quality gate enforcement

SCA

Snyk dependency scan — blocks known vulnerable packages

Container Scan

Trivy image scan — CRITICAL + HIGH severity findings block the pipeline

Approval Gate and Push to manifest repo

Approve after error-check -> bot updates values-prod.yaml -> ArgoCD auto-syncs

Merge → create GitHub Release tag (v1.0.1) → build image with semantic version → push Docker Hub → ArgoCD auto-syncs ✅

01

Approval Gate

GitHub Environment: `production` — manual approval required before any deployment proceeds

02

Build & Tag

Semantic versioning (`v1.0.1`) — Docker build + push to Docker Hub

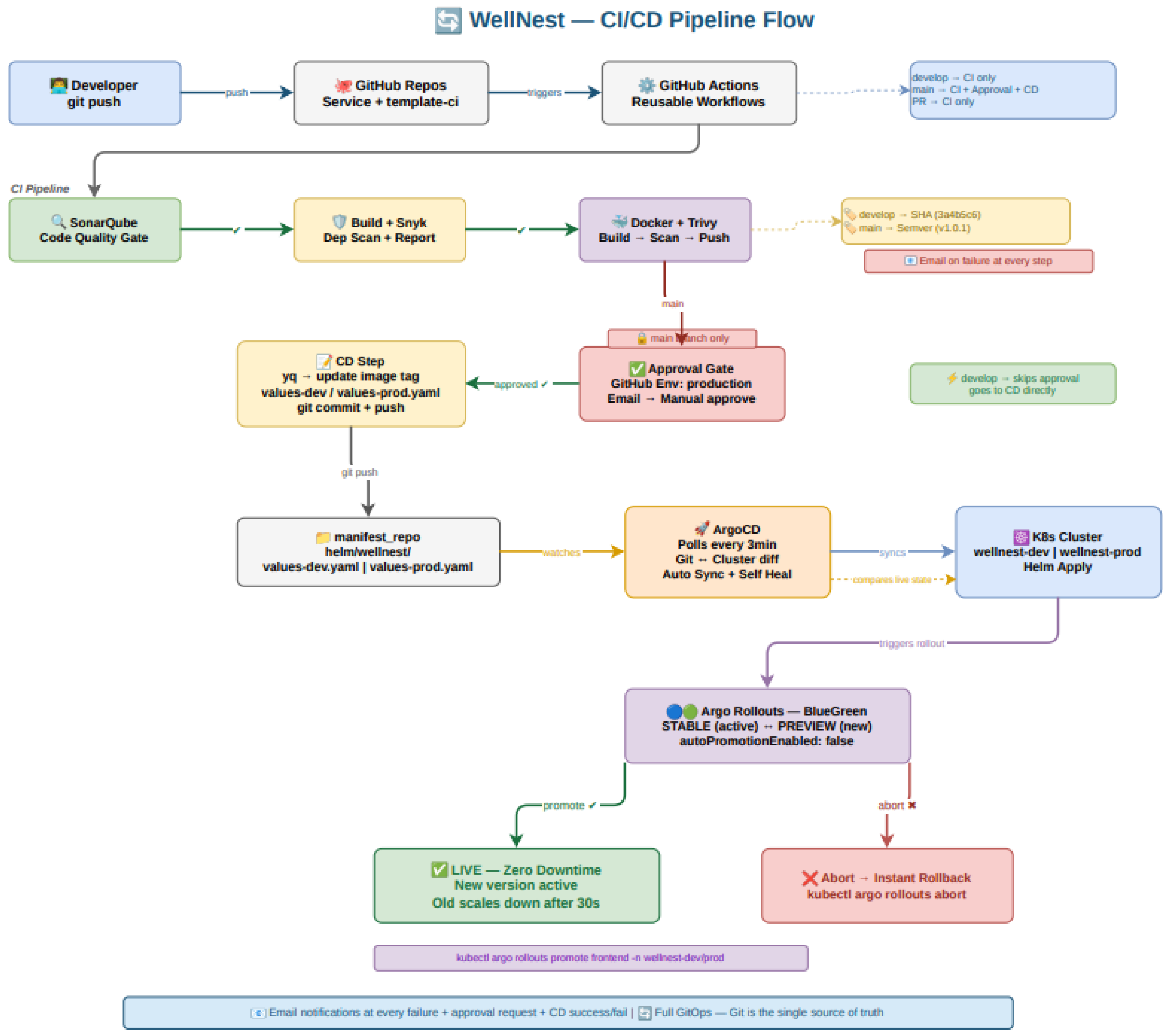
03

GitOps Update

CD bot updates `values-prod.yaml` → ArgoCD detects drift and syncs cluster

CI/CD End-to-End Flow

wellnest-ci — A reusable GitHub Actions workflow library shared by all service pipelines, ensuring consistent security gates and build steps across every microservice.



Pipeline Execution & Release Workflow

Dev Pipeline — Develop Branch

Trigger

PR to `develop` branch opens pipeline

Security Gate

SonarQube SAST · Snyk SCA · Trivy container scan

Merge & Build

Reviewer approves → image built as `dev-{SHA}` and pushed

GitOps Sync

ArgoCD detects `values-dev.yaml` update → auto-deploys to dev

Prod Pipeline — Release Workflow

PR to Main

Same full security gate runs — no image built on merge

GitHub Release

Developer creates release tag (e.g. `v2.0.0`) to trigger prod pipeline

Approval Gate

Manual approval via GitHub Environment: `production`

Deploy to Prod

Image tagged `v2.0.0` pushed → ArgoCD syncs `wellnest-prod`

Key Takeaways & Architecture Summary

Cloud-Native by Design

4 containerized services, multi-stage Docker builds, non-root execution, minimal Alpine base images.

Production-Grade Kubernetes

Namespace isolation, HPA autoscaling, StatefulSet MongoDB, NFS persistence, Gateway API routing.

Shift-Left Security

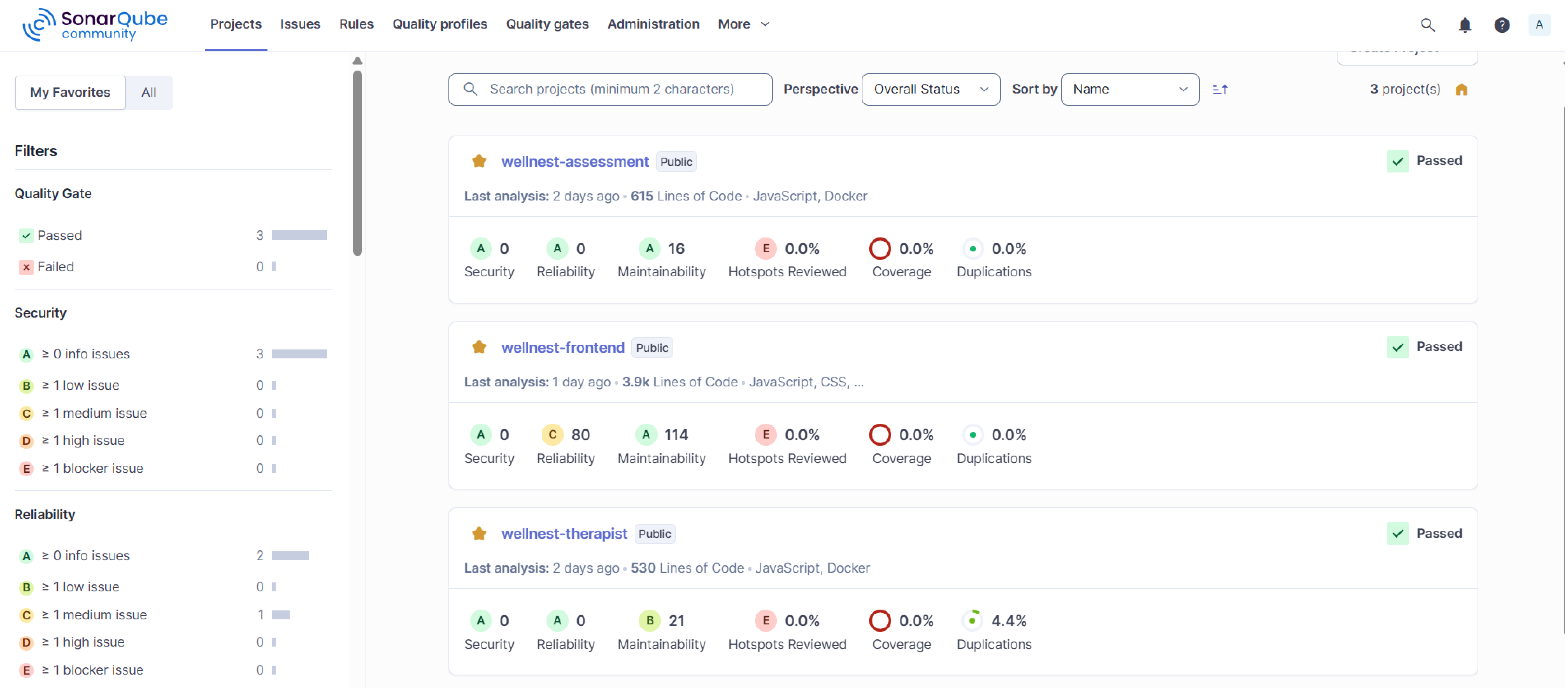
SAST, SCA, and container scanning on every PR. CRITICAL/HIGH findings block the pipeline before merge.

True GitOps Delivery

ArgoCD drives all deployments from Helm values. No manual `kubectl apply` — cluster state is always declarative.

> WellNest demonstrates a complete DevSecOps lifecycle — from code commit to production deployment — with security, observability, and infrastructure-as-code built in at every layer.

SonarQube Analysis Dashboard



Synk Report

snyk

Snyk test report

April 25th 2026, 10:22:42 am (UTC+00:00)

Scanned the following path:

- /home/runner/work/auth-service/auth-service/package-lock.json (npm)

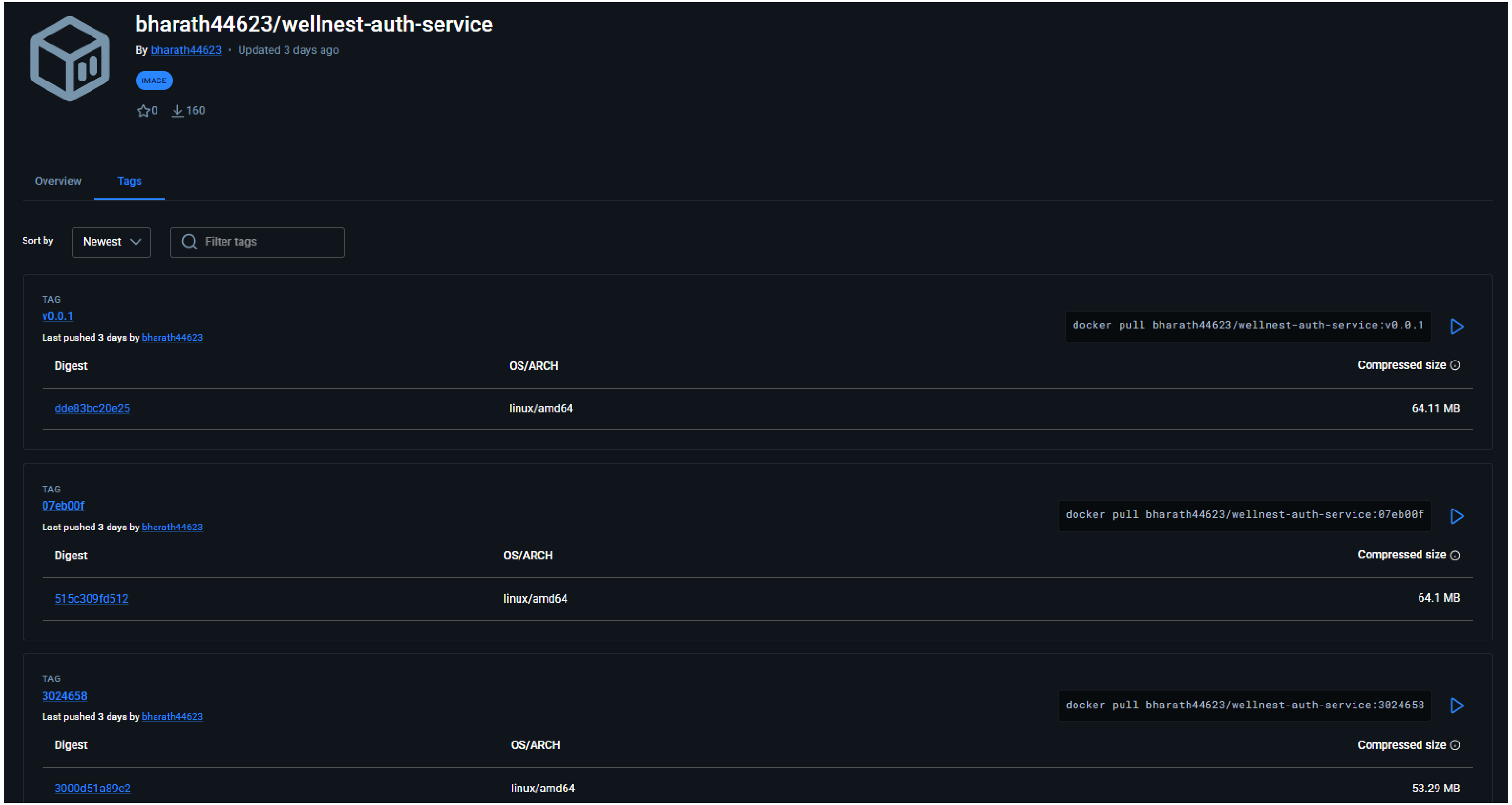
0 known vulnerabilities | 0 vulnerable dependency paths | 113 dependencies

Project	wellnest-auth-service	Path	/home/runner/work/auth-service/auth-service
Package Manager	npm	Manifest	package-lock.json

No known vulnerabilities detected.

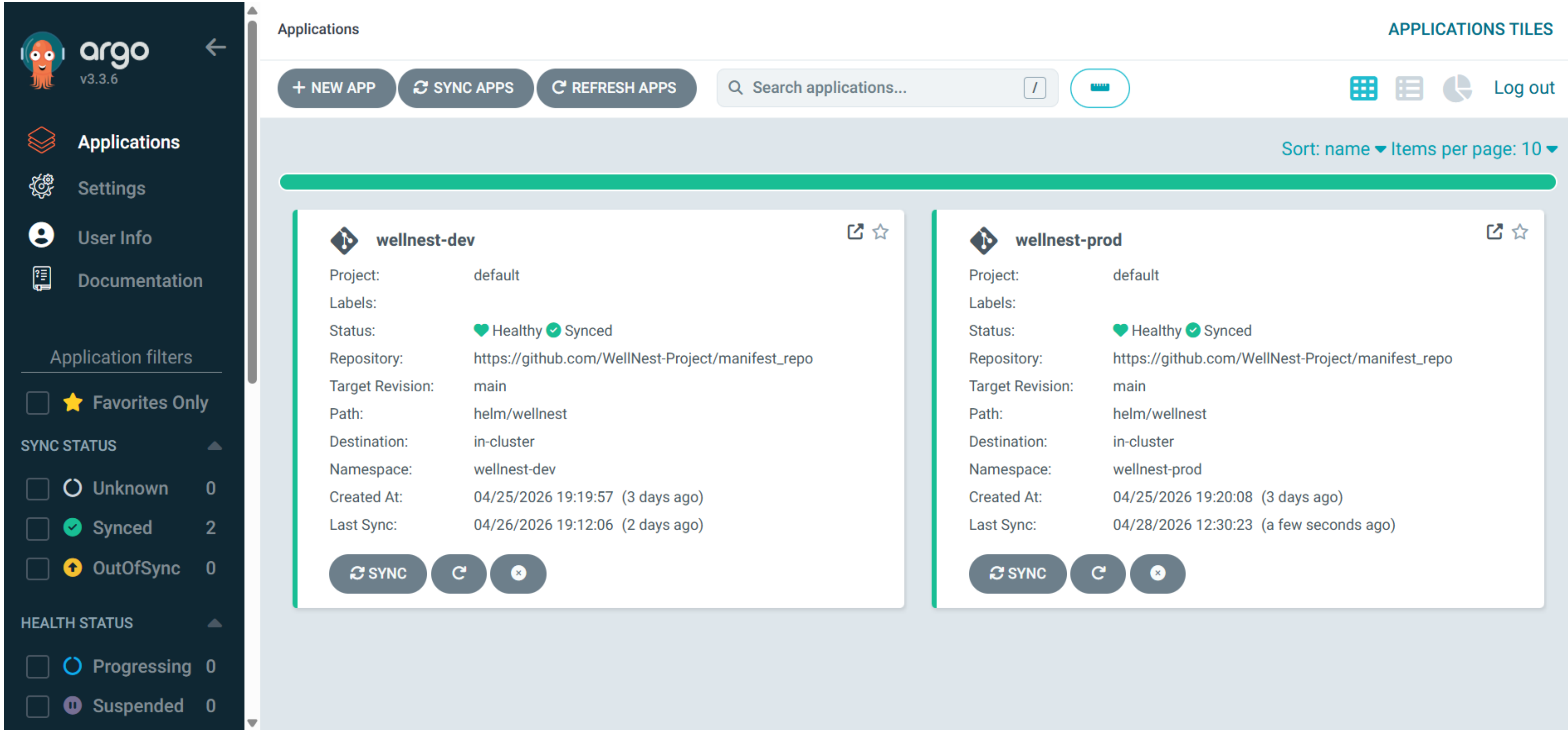
Docker Hub Repository Tags

bharath44623/wellnest-auth-service



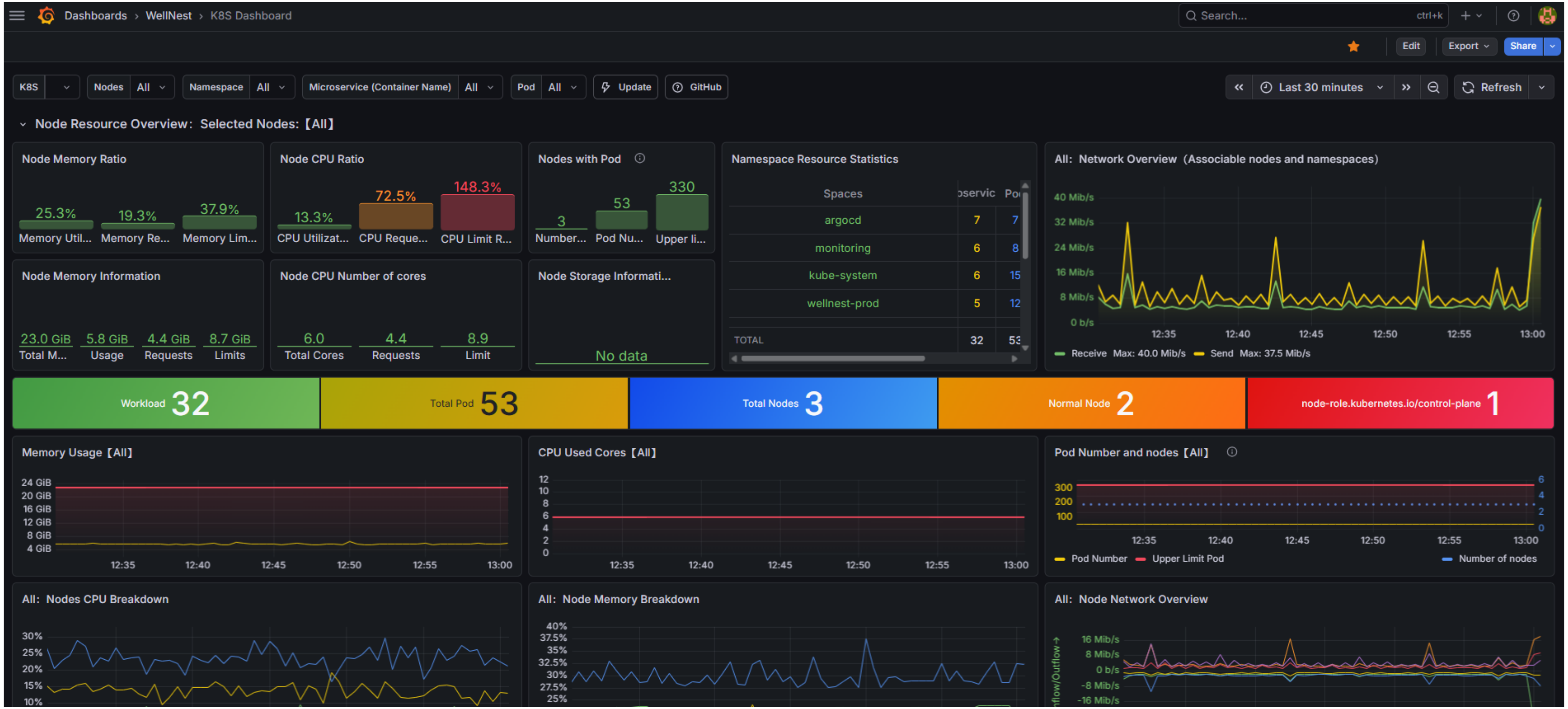
GitOps with ArgoCD

ArgoCD manages deployments using the app-of-apps pattern. selfHeal + prune enabled — cluster state always matches Git. develop branch → DEV cluster. main branch → PROD cluster.



Monitoring — Prometheus & Grafana

Prometheus + Grafana deployed on both dev and prod clusters. Exposed via HAProxy on NodePorts 32000 (Grafana) and 32001 (Prometheus). Custom WellNest dashboards track service health, pod metrics, MongoDB status, and HPA scaling.



Thank You

GitHub Repository

- WellNest-Project — Organization (https://github.com/WellNest-Project)

Live URLs

- Production App (http://wellnest-project.online/)
- Development App (http://wellnest-project.online:8081)
- ArgoCD UI (http://wellnest-project.online:8080)
- Grafana (http://wellnest-project.online:3200)
- Prometheus (http://wellnest-project.online:9090)